Delegate 총정리

TMulticastDelegate 설계와 순회 중 안전 처리

Tech Lab 9주차 · Game Jam #3

1. Delegate란?

> **일이 일어난 쪽이, 누가 반응할지 모르는 채로 알리는 도구**

- 알리는 쪽: `Broadcast`만 호출
- 듣는 쪽: 스스로 `Add`로 등록
- 서로를 몰라도 연결됨 → 엔진 코드가 게임 로직을 몰라도 됨

2. 왜 필요한가

Delegate 없이	문제
직접 호출	엔진 컴포넌트가 점수·사운드·게임 클래스를 알아야 함
상속	반응마다 서브클래스 필요. Lua는 C++을 상속할 수 없음
폴링	매 프레임 헛검사

Delegate를 쓰면 엔진은 "겹쳤다"만 알리고, 반응은 C++ 액터나 Lua가 각자 정한다.

3. 구조 한눈에

```
AGameMode
 └ OnScoreChanged : TMulticastDelegate<int32>
     └ InvocationList
         ├ [0] { Handle 1, Owner UI,   Handler, bPendingRemove }
         └ [1] { Handle 2, Owner 업적, Handler, bPendingRemove }
```

- Delegate는 **알리는 객체의 멤버 변수**. 객체마다 따로 있다
- 칸 하나 = 등록 하나 (`FDelegateInstance`)

4. 구성 요소

이름	역할
`FDelegateHandle`	등록 번호. 1부터 발급, 0은 무효, 재사용 안 함
`Handler`	실제로 불리는 함수 (`std::function<void(Args...)>`)
`Owner`	등록한 주체. `RemoveAll` 기준. 비교만 하고 역참조 안 함
`bPendingRemove`	해제 표시. Broadcast는 건너뛰고 Compact가 실제로 뺌
`InvocationList`	등록 목록 (`TArray`)
`BroadcastDepth`	실행 중인 Broadcast 중첩 수. 0이 아니면 삭제를 미룸

5. API

함수	하는 일
`Add(Fn, Owner)`	람다·함수 등록 → Handle 반환. Owner 기본값 없음
`AddDynamic(Obj, &T::F)`	멤버 함수 등록. Obj가 자동으로 Owner
`Broadcast(args...)`	등록 순서대로 전부 호출
`Remove(Handle)`	하나 해제
`RemoveAll(Owner)`	그 Owner 것 전부 해제. nullptr이면 무시
`Clear()`	전부 해제 (재시작)
`IsBound()`	표시 안 된 구독자가 있나

6. 실행 흐름

```
BeginPlay  → Add / AddDynamic          (저장만, 실행 X)

겹침 발생  → Broadcast(Sphere, Player)
               ├ Handler[0] 실행 → 리턴
               └ Handler[1] 실행 → 리턴    (동기, 순서대로)

EndPlay    → RemoveAll(this)
```

Broadcast는 저장된 함수를 순서대로 부르고 각각 끝날 때까지 기다리는 평범한 for문이다.

7. 위험 ① 순회 중 해제

상황: 핸들러 안에서 Destroy → EndPlay → `RemoveAll`

바로 지우면

```
[A, B*, C]   A 실행 중 B 제거
→ [A, C]     C가 1번으로 당겨짐
Index 2      Count=3 → 범위 밖
```

지금 방식

```
B.bPendingRemove = true
Index 1  → 표시됨, continue
끝       → Compact()로 제거
```

Compact는 **뒤에서부터** 지운다. 앞에서부터 지우면 당겨진 원소를 건너뛴다.

8. 위험 ② 재진입

상황: `OnScoreChanged` 핸들러가 다시 점수를 바꿈 → 같은 Delegate를 다시 Broadcast

```
바깥 Broadcast (Depth 1)  Index 0 실행 중...
  └ 안쪽 Broadcast (Depth 2)  처음부터 끝까지
    끝: Depth 2→1  → Compact 안 함
바깥 계속 Index 1, 2 ...
끝: Depth 1→0  → 여기서 Compact
```

- 안쪽이 끝날 때 정리하면 아직 도는 바깥 순회가 깨진다
- 그래서 bool이 아니라 **수**로 세고, 0일 때만 Compact

9. 위험 ③ 순회 중 추가

```
const int32 Count = InvocationList.Num();   // 시작할 때 고정
for (int32 Index = 0; Index < Count; ++Index)
```

- 새 항목은 **다음 Broadcast부터** 불린다
- 무한 루프 방지, "이벤트 시점의 구독자에게만 알린다"는 규칙
- 이번 이벤트에도 반응해야 하면 **등록하는 쪽이 직접 한 번 호출**
- 순회 중엔 Remove가 배열을 줄이지 않으므로 Count가 범위를 넘지 않는다

10. 위험 ④ 재할당 (핵심)

```
실행 중 람다의 this → X[1] { Self, Goal }
        |
  Add → 배열 이사 X → Y, X 해제
        |
  Self 읽기 → 해제된 X[1]을 읽음 X
```

- `Self` 값(AGameMode 주소)은 멀쩡하다
- 사라진 건 그 값을 **담은 칸** X[1]
- 실행 중인 함수는 도중에 Y를 알 방법이 없다

해결: 실행 전에 복사

```
HandlerType HandlerCopy =
    InvocationList[Index].Handler;
HandlerCopy(args...);
```

- 람다가 처음부터 **스택의 사본**을 기준으로 실행
- 배열이 이사 가도 무관

11. 재할당: 캡처만의 문제가 아니다

```
struct AchievementLambda
{
    AGameMode* Self;  // 캡처
    int32 Goal;
    void operator()(int32);
};
```

```
struct FEnemy
{
    int32 HP;
    void Split()
    { World->Add(...); HP /= 2; }  // X
};
```

- 람다 = 멤버 변수(캡처) + 멤버 함수(본문)를 가진 객체
- 근본 원인: **실행 중인 멤버 함수의 this가 가리키는 객체가 사라짐**
- 재할당을 일으킨 Add **뒤에서** 멤버(캡처)를 읽을 때만 터진다 → 재현이 어렵다

12. Owner 규칙

상황	Owner
`this`나 객체 포인터를 캡처한 람다	**반드시 넘김**
캡처 없음, 값만 캡처	`nullptr` 가능
Handle을 직접 관리	`Remove(Handle)`로 해제

- Owner 기본값을 없애 매번 의식하고 쓰게 함
- `RemoveAll(nullptr)`은 무시 → Owner 없는 등록이 한꺼번에 지워지는 것 방지
- EndPlay에서 `RemoveAll(this)`가 기본 규칙

13. 언리얼과 비교

	언리얼	우리
종류	Single, Multicast, Dynamic, Event, Sparse	Multicast 하나
바인딩	`AddUObject` (약한 참조), `AddRaw`, `AddLambda`	`Add`, `AddDynamic` (보호 없음)
순회 중 해제	표시 후 Compact	같음
순회 중 추가	다음 Broadcast부터	같음
위치	`Runtime/Core/Public/Delegates`	`Runtime/Core/Delegates`

14. 한계와 다음 단계

한계

- Owner가 RemoveAll 없이 사라지면 위험
- 약한 참조(`TWeakObjectPtr`) 아직 없음
- 사용처가 없어 템플릿 본문 컴파일 검증 전

다음 단계

- `UPrimitiveComponent` 에 `OnComponentBeginOverlap` / `EndOverlap` / `Hit`
- 겹침 검사에서 Broadcast
- Lua 스크립트를 `Add(…, Owner)` 로 연결, EndPlay에서 RemoveAll